NMR Analysis Suite

A Custom Python Application for T1 Relaxation and CSP Waterfall Plot Creation

Built with: Python · PyQt 6 · NMRglue · SciPy · Matplotlib · Numpy

What We'll Cover

Presentation Roadmap

01

Why Build This?

Problems with paid alternatives, versatility of NMRglue and how Bruker files are converted to usable data.

02

Tech Concepts and Software Architecture

IDEs, Git, Repositories. How and why the program is organized and how the modules connect. UI/UX decisions.

03

The Math Involved

Exponential decay fitting of T1 data, and phasing of peaks on 2d spectra.

04

Reliability & Robustness

How the software handles bad data and edge cases.

05

Future Directions

How to access the codebase, how future contributors can make changes, possible additions.

06

Tech Demo

Live walkthrough of installation, setup, and T1/CSP modules.

Definitions and Context

Relevant Information

IDE - A software that provides an environment for writing, running and debugging code	**Python Module -** A single .py file that contains code for one specific job	**Git -** A version control system that tracks changes made to the codebase overtime
GitHub - A website that hosts Git repositories, allowing sharable code across machines	**Repository -** The folder that contains all of a project's files and change history	**Fork -** A personal copy of a repository that allows editing without affecting the original.
Branch - A parallel version of the code within a repository, used to develop without damaging the main version	**Import -** A line of code that loads an external library into the code, giving it access to prewritten tools	**Library -** A collection of pre-written code that already solves common problems

Section 01

Why build this?

Problems With Alternatives

The Problem with Existing Software

- TopSpin requires a commercial Bruker License - Costing $698.50 anually for the Mnova Suite
- Can't access outside of the lab environment or specific machine.

The Bruker Format Problem

- All data from Bruker instruments come out as a proprietary binary format
- Normal tools such as Excel or MATLAB cannot read these without a dedicated parser

Why and What is NMRglue?

Section 01 - Why Build This?

What is NMRglue?

- NMRglue is a free, open source Python library built to parse almost every mainstream proprietary NMR format.

Why NMRglue?

- It allows direct access to raw data – spectra, delay list, acquisition parameters and more.
- It can be built on top of, meaning the entire pipeline is customizable.

How Bruker Data is Loaded

Section 01 - Why Build This?

The Pipeline

- Bruker instruments save data in a proprietary binary format – it can't be read without special tools.
- NMRglue translates the binary into a standard python array.
- Along with the raw data, it reads the experimental parameters embedded in the file such as the pulse program, nucleus, spectral width and field strength.

Building the ppm Axis

- Raw data comes out as an array, this means it has no chemical shift information attached to it.
- NMRglue has a built in tool for this, but it can fail when working with the data that we're working with.
- To prevent this failure we directly translate the procs file stored in the dataset and calculate the axis ourselves.

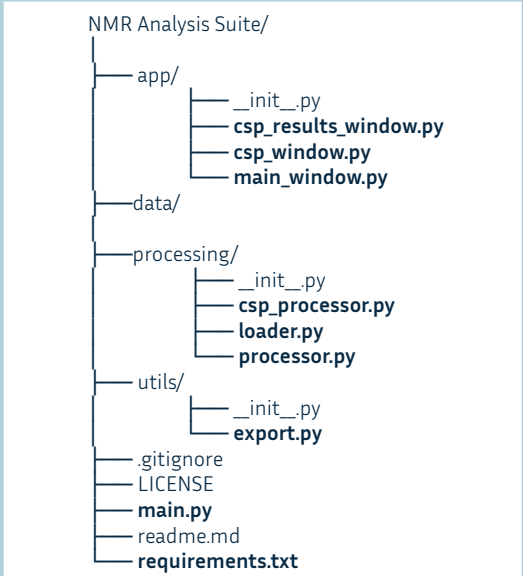
```
270        # ppm axis: left edge = OFFSET, right edge = OFFSET - SW_p/SF
271        # (ppm decreases left → right in NMR convention)
272        sw_ppm = sw_p / sf
273        ppm    = np.linspace(offset, offset - sw_ppm, si)
274        return ppm
```

Section 02

Tech Concepts and Software Architecture

Program Architecture

Section 02 - Tech Concepts and Architecture



app/ - UI layer, everything that the user sees and interacts with
data/ - was used for test data, will be removed
processing/ - All computational and data handling
utils/ - shared tools by both modules
main.py - startup point, launches the application

Design Philosophy - I tried to make the UI and math completely separate, so that the processors have no knowledge of the buttons, windows or displays. This means that logic can be tested, or replaced without touching the interface.

I still need to reformat the csp_window to remove math logic

UI/UX Design Decisions

Section 02 - Tech Concepts and Architecture

Designed for Specific Uses - This program was designed for specific use cases, such as building waterfall plots with concentration and T1 data which is a large limitation compared to something like top-spin, but the specific things it does do it does really well.

T1 Module

- ❏ Auto-detects correct experimental folder.
- ❏ Drag to select integration window
- ❏ Fit results and warnings displayed immediately

CSP Module

- ❏ Supports loading multiple Bruker spectra in sequence.
- ❏ Automatically orders by lowest concentration to highest
- ❏ Selection of spectra, allowing scrolling and phasing of each one individually.

Export

- ❏ The T1 Module outputs all parameters to a csv file after fitting.
- ❏ The CSP module outputs the graph window to a black/white png file.

Module Connections and Data Flow

Section 02 - Tech Concepts and Architecture

Launcher

- main.py starts application and presents two options
- Selecting T1 opens main_window.py, selecting CSP opens csp_window.py
- Neither module loads on startup unless the user selects it.

T1 Flow

- Main_window receives the folder path from the user
- Passes it over to loader.py which reads bruker files and returns data array
- main_window.py passes the data and integration window to processor.py which returns fit result.
- Result is displayed in main_window.py and written to disk by export.py

CSP Flow

- csp_window.py receives multiple folder paths, one per spectrum.
- Each is individually passed to loader.py, which returns 2D spectrum and ppm axis.
- csp_window.py passes all spectra to csp_processor.py which returns results.
- export.py writes png to disk

Section 03

The Math Involved

T1 Curve Fitting

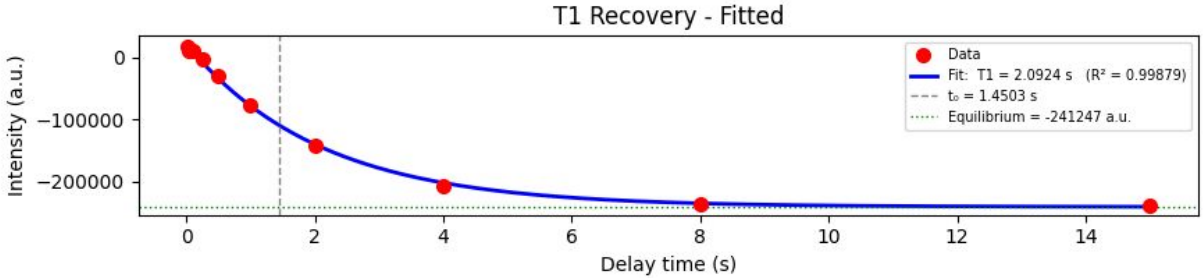
Section 03 - Math Involved

The T1 curve is fitted using the equation: $I(t) = A(1 - 2e^{(-t/T1)}) + C$

A is the equilibrium intensity, representing the fully relaxed state.
T1 is the relaxation time we're solving for. How long it takes for the spin to return to equilibrium.
C is the baseline offset, it accounts for the drift that occurs in the inversion pulse.

The program uses a function from SciPy called **scipy.optimize.curve_fit**. It starts with the initial T1 value, draws the curve, measures how far it is off from the data points, then adjust and tries again. It does this iteration thousands of times until it finds the values of A, T1, and C to make it as accurate as possible.

If we take our starting equation and set I(t) = 0 and solve for t, we get t = T1 * ln(2). This means the curve is always crossing zero at 0.693 * T1. The software finds where the data crosses zero and works backwards to find a meaningful starting point.



T1 Curve Fitting - pt 2

Section 03 - Math Involved

Polarity Detection - Before any fitting happens, the software checks whether your data is in standard orientation or phase flipped by Bruker's built in processing. It does this by comparing the first and last points of the trajectory

If the last number in the array is greater than the first then the curve is as expected, if not it's multiplied by -1 to correct then flipped back for the display.

R^2 and Residuals - R^2 is calculated as 1 - SS_res / SS_tot. Where SS_res is defined as the total squared difference between the actual data points and the fitted curve. SS_tot is defined as the total squared difference between the actual data points and their mean. (How spread the points are)

If the total curve fits perfectly SS_res is zero so $R^2 = 1$. This is exactly the same calculation that excel does for R^2.

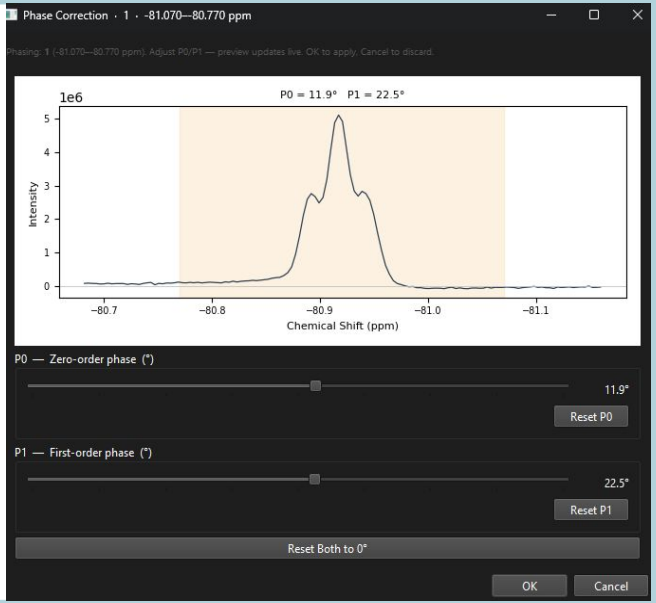
Phase Correction in the CSP Module

Section 03 - Math Involved

Why Phasing is Necessary - When a spectrum comes off the spectrometer, the real and imaginary components of the signal are mixed together, distorting it to be asymmetric or partially negative.

P0 - Zero Order Phase - A constant rotation applied to the entire spectrum, corrects for a timing offset between the pulse and the start of data collection.

P1 - First Order Phase - A rotation that varies linearly across the spectrum, corrects for different frequencies evolving at different rates during the data acquisition.



Phase Correction in the CSP Module - Pt 2

Section 03 - Math Involved

How it's Applied in the Code - The software reads the 1r and the 1i files from the experiment folder. On it's own neither is a clean peak, together they form a complex number at every point. To phase correct it you multiply the complex signal by exp(iφ) where φ is the phase angle or the amount of rotation being applied, and i is the square root of -1.

Why it works - After the rotation, the signal that was split between the real and imaginary channels is now aligned so the peak sits on the real channel. Discard the imaginary part and keep the real component and this is the phased spectrum.

Section 04

Reliability and Robustness

T1 Module - Reliability

Section 04 - Reliability and Robustness

Strengths:

- ❏ Multi-seed optimizer reduces chance of converging on the wrong answer for the T1 curve
- ❏ Polarity detection handles phase-flipped data without analyst intervention
- ❏ Data warns and tells the analyst when their delay list is too short to produce a reliable T1
- ❏ Graceful error handling and fallback from processed to raw data if a folder is missing in the experiment folder

Important Limitation - Has not been tested against a known T1 value for accuracy.

CSP Module - Reliability

Section 04 - Reliability and Robustness

Strengths:

- ❏ Phase correction is applied per-spectrum within only the peak region.
- ❏ Baseline correction is iterative, meaning it doesn't corrupt baseline estimate from bruker files (easily revertable)

Limitations:

- ❏ If two peaks are close together, Gaussian fitting window may capture both.
- ❏ Some spectra don't normalize cleanly if signal is outside of normal range.
- ❏ Manual phase correction inherently introduces error
- ❏ Not validated against known values

Section 05

Future Directions

Possible Improvements

Section 05 - Future Direction

Validation:

- ❏ Running the T1 module against samples with known T1 values, same for CSP.

CSP Module:

- ❏ Automated phase correction, handling overlapping peaks better, Kd determination.

General:

- ❏ Maybe a settings or config file in order to edit factors like how aggressive the parameters are?
- ❏ Refactoring of codebase for organization, some math is outside of processing files.

Section 06

Tech Demo

Questions and Final Thoughts

Tech Demo using Conda